

Dossiê do Projeto: Sistema de Vendas Mobile

Compilado de Solicitações do Usuário e Engenharia de Prompt Final (Antigravity)

1. Histórico de Requisitos Solicitados

Este documento consolida todas as diretrizes, restrições e requisitos de negócio fornecidos para a concepção e estruturação do aplicativo de portfólio. As solicitações foram divididas em três blocos principais de evolução:

Bloco 1: Escopo Geral e Arquitetura Base

- **Tecnologia:** Desenvolvimento híbrido utilizando o framework Ionic com Angular.
- **Persistência de Dados:** Utilização do banco de dados relacional local SQLite.
- **Módulo de Autenticação:** Tela inicial de Login exigindo Usuário e Senha.
- **Módulos de Cadastro:** Telas para inserção de novos Usuários, novos Clientes e novos Produtos. O cadastro do produto é o responsável por inicializar e alimentar o estoque no banco de dados.

Bloco 2: Lógica Transacional e Regras de Negócio

- **Tela de Vendas:** Interface que correlaciona os clientes e os produtos previamente cadastrados.
- **Efetivação da Venda (Regra de Negócio):** Ao concluir a venda, o sistema deve obrigatoriamente executar duas ações automáticas:
 1. Remover a quantidade correspondente do produto do estoque (baixa de estoque).
 2. Gerar um documento ou registro para o módulo Financeiro na categoria de "Contas a Receber".
- **Módulo de Relatórios:** Um campo ou tela dedicada para a geração de relatórios com o histórico completo das vendas realizadas.

Bloco 3: Restrições Técnicas Avançadas e Identidade Visual

- **Garantia de Unicidade de Cadastro:** O cliente deve possuir uma chave de acesso única obrigatória (como o CPF), configurada de modo que o sistema impeça estritamente a duplicidade de cadastros de clientes.
- **Alta Capacidade de Armazenamento:** O banco de dados SQLite deve ser projetado para suportar grandes volumes de dados (armazenar múltiplos objetos em tabelas de usuários, clientes e produtos com alta performance).
- **Identidade Visual / Cores do Aplicativo:** O layout visual e a estilização dos componentes do app devem ser baseados exclusivamente em variações e tons de verde.

2. Prompt Final Consolidado (Enviado ao Antigravity)

Abaixo está registrado o prompt técnico final resultante da fusão de todas as especificações acima, utilizado para instruir a Inteligência Artificial a gerar o código-fonte correto do projeto:

Defina o papel de um Engenheiro de Software Sênior especialista em desenvolvimento híbrido com Ionic, Angular e banco de dados SQLite local. Preciso criar a estrutura base, o design de componentes e as páginas de um aplicativo de portfólio acadêmico de um Sistema de Vendas de Produtos.

O projeto deve seguir estritamente os seguintes requisitos tecnológicos, arquiteturais e de design:

- Framework: Ionic (versão mais recente) com Angular.
- Banco de Dados: SQLite local utilizando a abordagem recomendada pelo Ionic/Capacitor. O banco deve ser projetado para alta capacidade e performance, suportando grandes volumes de registros (milhares de usuários, clientes e produtos).
- Identidade Visual: O tema visual do aplicativo deve ser baseado estritamente em tons de verde (Green/Emerald/Forest), aplicando em botões, toolbars, cards e elementos ativos, mantendo um design limpo e moderno.

Por favor, gere a estrutura de código contemplando:

1. ARQUITETURA DO BANCO DE DADOS (Script de Inicialização e Performance):

Crie as tabelas SQLite aplicando as seguintes restrições e otimizações de índices para buscas rápidas em grandes volumes de dados:

- usuarios (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT, login TEXT UNIQUE, senha TEXT)
- clientes (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT, cpf TEXT UNIQUE, email TEXT, telefone TEXT) -- CPF como chave única, impedindo cadastros duplicados.
- produtos (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT, preco_venda REAL, quantidade_estoque INTEGER)
- vendas (id INTEGER PRIMARY KEY AUTOINCREMENT, cliente_id INTEGER, data_venda TEXT, valor_total REAL, FOREIGN KEY(cliente_id) REFERENCES clientes(id))
- itens_venda (id INTEGER PRIMARY KEY AUTOINCREMENT, venda_id INTEGER, produto_id INTEGER, quantidade INTEGER, preco_unitario REAL, FOREIGN KEY(venda_id) REFERENCES vendas(id), FOREIGN KEY(produto_id) REFERENCES produtos(id))
- financeiro_receber (id INTEGER PRIMARY KEY AUTOINCREMENT, venda_id INTEGER, data_vencimento TEXT, valor REAL, status TEXT, FOREIGN KEY(venda_id) REFERENCES vendas(id))

* Crie scripts de 'CREATE INDEX' para os campos 'cpf' em clientes, 'login' em usuarios e 'venda_id' nas tabelas filhas para garantir a escalabilidade do banco de dados grande.

2. SERVIÇOS (Services TypeScript):

- Implemente as funções CRUD com tratamento de erros. No cadastro de clientes, trate explicitamente a exceção de conflito caso o CPF já exista (Constraint UNIQUE), exibindo um alerta amigável de "Cliente já cadastrado".
- ATENÇÃO À REGRA DE NEGÓCIO DA VENDA: Crie um método transacional para efetivar a venda que:
 - a) Verifique se há estoque disponível do produto.
 - b) Insira o registro na tabela 'vendas' e os respectivos 'itens_venda'.

- c) Atualize a quantidade na tabela 'produtos' subtraindo os itens vendidos.
- d) Insira automaticamente um registro na tabela 'financeiro_receber' com o valor total da venda e status 'Pendente'.

3. TELAS E DESIGN (Layout com Tematização Verde):

Forneça o código das páginas utilizando componentes nativos do Ionic e inclua a customização do arquivo 'theme/variables.scss' alterando as cores '--ion-color-primary' e '--ion-color-success' para tons de verde corporativo/moderno.

- Tela de Login: Validação simples consultando a tabela de usuários.
- Tela de Cadastros: Formulários limpos para Cadastro de Usuário, Cadastro de Cliente (com validação e máscara de CPF) e Cadastro de Produto.
- Tela de Vendas: Interface de carrinho com seleção de Cliente, adição de múltiplos Produtos e validação de estoque antes de finalizar.
- Tela de Relatório: Listagem performática de histórico de vendas usando INNER JOIN.

Forneça as instruções de onde salvar cada arquivo e os comandos CLI do Ionic.